

场景化扩展评估轴：Adapter、配置与执行协议

日期：2026-09-05

状态：完整方案草案，供整体审阅；尚未进入实现。

范围：在 `llm\_probe` 内建设独立的扩展评估轴试验体系，生产继续使用原轴1、轴2。

1. 目标与已经对齐的方向

现有评测主要由能力兑现（轴1）与承载性归位（轴2）组成。新体系需要允许场景选择更多评估维度，同时允许不同评估方法拥有不同执行流程。用户配置不同描述并不足以表达所有方法差异。

已对齐的方向：

- 1. 一类轴对应一个 adapter，adapter 实现该类轴的评估方法。
- 2. 场景选择启用哪些轴，并为这些轴填写本场景的标准和可选资料。
- 3. 不同 adapter 可以采用不同流程：单次自主探索、多阶段、逐项核实、确定性程序检查等。
- 4. 新版轴1、新版轴2也在扩展体系内部实现，以复制既有逻辑为主。新版轴2消费新版轴1的结果，不借用生产结果完成自己的评估。
- 5. 新体系暂不启用生产接入，旧链路继续作为生产入口和比较基线。
- 6. 复用 Agno、LlmClient、资料检索及结构化校验等公共基础设施，不复制第二套底座。
- 7. 先验证新体系能容纳原有两种方法并保持效果，再增加或优化其他方法。

本文给出一套可以实施的协议建议。字段名、状态名、前端分区和调度细则是本次草案的具体提议，不代表此前已经逐项确认。

2. 概念与职责

- | 概念 | 定义 | 维护者 |
- | --- | --- | --- |
- | 轴类型 | 一种评估方法的稳定标识，如 `fulfillment`、`carryability` | 开发者 |
- | Adapter | 轴类型的代码实现及输入、配置、输出契约 | 开发者 |
- | 场景轴实例 | 某场景选择一个类型后创建的具体评估轴，包含业务定义和输入连接 | 资料页用户 |
- | 样本 | 本次评测使用的请求、已得到的回答及必要元信息 | 试验入口 |
- | 运行计划 | 根据场景启用项和数据连接生成的依赖图 | 运行器 |
- | 轴运行结果 | 单样本、单轴实例的一次执行状态和方法专属输出 | Adapter 与运行器 |

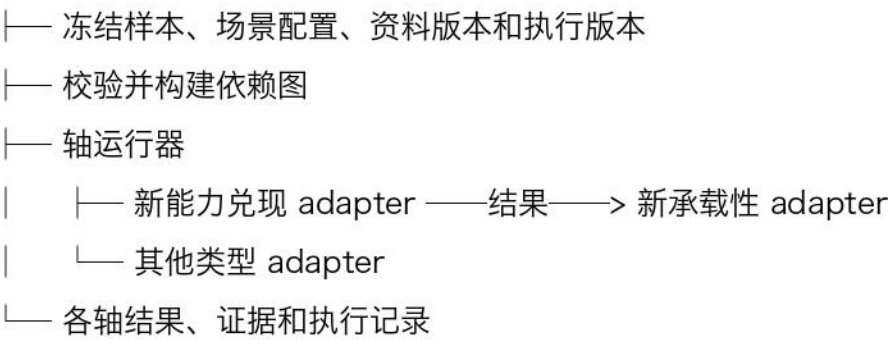
同一种 adapter 可以被多个场景使用。同一场景也可以创建多个同类型实例，例如使用整体评估方法分别评价流畅度和语气。

“类型相同”表示执行方法相同，不要求业务标准相同；“结果外层统一”不表示所有类型必须输出同一套枚举或分数。

3. 系统边界

```text

扩展评估轴试验入口



原生产轴1 → 原生产轴2 → 生产结果

↓

同一冻结数据上的新旧比较

公共基础设施：Agno / LlmClient / 资料工具 / 结构化校验 / 上下文记录  
...

扩展体系内部拥有配置加载、依赖调度、轴类型实现、输入构建和结果处理。它不调用生产轴1/轴2入口来冒充自己的 adapter 实现。

复制的是轴专属业务逻辑，公共工具继续复用。复制后两条执行路径的区别是生产基线与试验候选，不是兼容层；不新增 migration、隐式 fallback 或全局切换旧实现的包装器。本次不删除仍承担生产职责的旧链路。

## 4. 开发者如何定义轴类型

建议目录：

```
```text
impl/projects/llm_probe/eval_axes/
protocol.py          # 契约、运行状态、结果外层
registry.py          # 显式注册类型
store.py             # 场景配置读写与校验
runner.py            # 依赖、输入、预算、生命周期
adapters/
  fulfillment.py      # 新版轴1专属逻辑
  carryability.py     # 新版轴2专属逻辑
...```
```

方法复杂时允许在 adapters 内拆分专属模块。先检查项目已有 schema、表单和存储能力，再决定具体复用位置；不为这些文件名另造框架。

4.1 类型声明

字段 约束与用途
--- ---
`type_id` 全注册表唯一的类型标识，实例通过它选择实现
`title` 资料页类型名称
`description` 说明该方法评价什么及所需证据
`config_schema` 可填写的业务配置及必填项；从既有结构化模型能力生成
`input_schema` 有名字的输入端口、数据类型及必需性
`output_schema` 方法输出类型；用于验证及下游类型检查
`default_bindings` 创建实例时的预填连接建议，不是运行时自动猜测
`execution_limits` 该方法的一次轴运行总时限、LLM 调用和工具调用上限

代码版本不要求开发者每次手改一个版本号；试验记录用代码修订及实际相关文件指纹标识。配置结构若改变，旧配置必须明确报不符合当前类型，不能静默迁移。

注册表显式列出可用类型。资料页只能选已注册类型，不能提交任意 Python 路径或代码。

4.2 Adapter 接口

概念接口如下，最终类型表达优先沿用项目现有方式：

```
```python
applicable(inputs, config) -> Applicability
execute(inputs, config, runtime) -> TypedOutput
validate_result(output, inputs, config) -> None
...```
```

- `applicable`：在输入就绪后做确定性的适用性检查，返回是否适用及原因。不能靠额外 LLM 猜测是否值得运行；确实需要语义分析的适用性判断属于 execute 内部方法。
- `execute`：实现评估流程，可调用多次受控 LLM、工具和程序；不得直接调用其他轴实例。
- `validate\_result`：检查本方法的语义结构与完整性，例如期望是否遗漏、引用是否真实、枚举是否合法。
- schema 校验由公共层执行，不要求每个 adapter 重复实现。

校验失败如允许修复，修复循环由该 adapter 明确实现并计入同一总预算。公共层不额外套一个未知次数的“自动再试”，防止嵌套重试扩大成本。

### 4.3 首批两种类型

类型	业务配置	输入	输出及方法
--- --- --- ---			
`fulfillment`	`capability`；需要时提供与旧语义一致的 `show_schema`	`request`、`answer`、必要的样本证据元信息	保留轴1期望、assessment、整体判定语义及原校验/重问流程
`carryability`	`boundary`	新版 `FulfillmentResult`	保留逐期望核实、引用重试、carry 与 placement 映射以及完整性约束

复制前必须列出旧入口实际消费的上下文和字段。表中短名称不是允许丢弃旧逻辑所需证据；任何投影变化都要在新旧比较中解释。

承载性 adapter 只把待处理期望及边界证据交给承载性模型，不因收到轴1完整结果就把所有字段传入模型。

## 5. 用户如何配置场景轴

建议存储路径：`impl/data/llm\_probe/eval\_axes.json`。

按现有场景标识存储，不另建第二套场景命名。是否等同 capability\_ref 必须以项目实际解析结果为准，不直接假设二者相等。

### 5.1 实例字段

字段	必填	来源与含义
--- --- ---		
`id`	是	新建时生成，场景内唯一；连接引用使用它
`type`	是	用户从类型下拉选择
`title`	是	本场景内的显示名称
`enabled`	是	是否纳入扩展评估试验，新增默认 false
`config`	是	按类型结构填写标准、资料引用等
`inputs`	是	输入端口到数据来源的绑定
`after`	是	默认空；只要求等待结束、不读取数据的顺序依赖

不设置一个全场景通用 rubric 继承层。第一版各场景独立配置，避免默认配置、覆盖配置和资料来源难以追踪。

### 5.2 完整配置示例

以下为 JSON 数据的 YAML 展示形式；该例表示用户已选择在试验中运行两个轴，不表示生产启用：

```
```yaml
scenarios:
  policy_search:
    axes:
      - id: fulfillment
        type: fulfillment
        title: 能力兑现
        enabled: true
```

```
config:
  capability: |
    根据用户问题回答政策内容，保留适用条件和例外。
    评估说明见 {material://llm_probe/policy-capability}
inputs:
  request:
    source: sample
    port: request
  answer:
    source: sample
    port: answer
  context:
    source: sample
    port: context
after: []

- id: boundary
  type: carryability
  title: 承载性
  enabled: true
  config:
    boundary: |
      本服务覆盖公开政策问答，不支持个案审批。
      边界说明见 {material://llm_probe/policy-boundary}
  inputs:
    fulfillment:
      source: axis
      axis_id: fulfillment
      port: output
  after: []
...
```

采用结构化 binding，避免运行时解析任意点路径或表达式。第一版只提供 sample 的已声明端口和各轴的 `output` 端口；不支持任意对象路径、模板脚本或 SQL 式选择器。

5.3 资料页填写位置

```
```text
资料页 → 选择场景 → 扩展评估轴（试验）
 轴列表：名称 / 类型 / 是否参与试验
 添加轴：选择类型 → 生成实例
 编辑轴：
 基本信息：名称、类型说明
 评估定义：按 config_schema 提供字段
 输入与依赖：按 input_schema 提供数据来源下拉
 高级顺序：after 多选
 运行限制：显示 adapter 的限制，本版不提供任意调参
...
```

文本描述复用自由资料引用插入能力。轴1和轴2没有任意刻度编辑器；后续整体评估 adapter 可以声明 instruction、scale 等字段，每个刻度需要判定含义。

首版只使用成熟的现有控件组合，不建设任意表单设计器。资料页明确提示：“当前仅供扩展评估试验，生产仍使用原轴1、轴2。”

从现有配置准备试验时进行显式复制，记录来源和内容指纹。此后试验读取自己的快照，不在执行时自动回退读取旧 capability 或 boundary。

## 6. 连接、执行位置与信息依赖

### 6.1 三种关系

- | 表达 | 含义 |
- | -- | -- |
- | inputs 绑定 sample 端口 | 样本可用即可准备执行 |
- | inputs 绑定 axes.X.output | 依赖 X 的有效输出，且只读取该端口 |
- | after 包含 X | 等待 X 进入终态，不获得 X 的任何数据 |

“after”的“结束”包括成功、不适用、失败、阻断或取消。需要 X 成功结果的关系必须使用 inputs，不允许用 after 暗示成功依赖。

保存时拒绝悬空轴 ID、不兼容类型、缺少必需输入、自依赖和循环。启用轴不得依赖禁用轴，用户必须同时启用上游或修改连接。删除被引用轴前必须先调整引用，不能自动断链。

默认连接只有唯一匹配候选时才预填；有多个候选时由用户选择。预填后保存明确来源，不在每次运行重新推断。

### 6.2 调度

内部使用最小 DAG 调度，不提供任意工作流语言或可视化 DAG 编辑器。图的边来自 inputs 与 after，展示顺序不参与调度。

样本准备好后，无上游依赖的轴可以开始；输入依赖满足且顺序依赖结束的轴进入适用性检查。独立轴可在现有并发限制内并行，不无限扩容。

新版轴2依赖新版轴1。表达质量等独立类型可以只读取样本，与新版轴1并行；不再统一写死“全部扩展轴必须等轴1”。

### 6.3 输入权限与阶段隔离

运行器构造限定输入快照，不把整个 RunTrace、全局 run 或所有其他轴结果默认暴露给 adapter。确有必要的 trace 字段通过 sample.context 的明确类型提供，并与旧证据视图对齐。

轴内不同阶段可以进一步投影输入。需要“先定标准再看回答”的方法，应以阶段信息隔离实现，不能只依赖 prompt 指令。

自由资料在试验开始时解析引用并保存内容快照。内联文本和检索工具读取同一快照，而不是仅记录哈希后继续查询可能变化的原文件。缺失引用在运行前报配置错误；空检索结果不等于事实为假。只给 adapter 注入当前配置允许的资料集合。

Adapter 是受信任项目代码；此处限定输入是运行协议和可验证工程边界，不声称 Python 进程内存在对恶意 adapter 的安全沙箱。

## 7. 执行环境、自主性与预算

runtime 提供受控 LLM 调用、选定资料工具、过程记录、取消信号和预算检查。各 adapter 使用这一入口复用现有 LlmClient，不直接新建绕过记录的 Agno Agent。

方法可以自主决定搜索词、资料读取顺序、是否进一步查证；不能运行中修改配置、刻度含义、允许来源、汇总规则或自己的预算。

一次轴运行的预算包括所有阶段、调用和修复重试。工具上限按真实执行次数计数，LLM 调用按实际模型请求计数；模型路由端点重试也必须计入，而非仅计算 complete\_json 的外层次数。需确认 Agno/路由现有可观测点，并在实际发出调用前阻断超额请求。

首批复制 adapter 的限制以旧配置和旧实际执行方式为依据确定，不凭空为所有方法指定相同次数。新框架总预算约束若改变旧行为，必须单独记录为框架差异，不算作“完全等价复制”。

总时限从轴进入 running 开始计算，排队时间单独记录；可另有试验运行级截止时间。调用超时不得超过本轴剩余时间。取消或超时后停止安排新阶段，拒绝迟到结果发布；正在执行的同步调用若不可立即中断，记录真实停止时间，不伪称已经杀死调用。

## 8. 生命周期与结果协议

### 8.1 执行状态

状态	含义
---	---
`disabled`	该实例未纳入计划，不占执行资源
`waiting`	等待输入、上游或并发资源
`running`	正在适用性检查或执行
`succeeded`	方法输出通过公共及专属校验
`not_applicable`	输入有效，但方法明确判断该样本不适用
`blocked`	所需上游未产出有效输出
`failed`	调用、校验或执行失败；reason_code 区分超时和预算等
`cancelled`	用户或运行级取消

模型认为证据不足属于方法的业务输出，可处于 succeeded 状态，例如新版轴1的 not\_evaluable、新版轴2的 undecidable；不能用它掩盖基础设施失败。

上游 not\_applicable 没有方法输出时，要求其 output 的下游进入 blocked。承载性在“轴1成功但没有待归位期望”时自身返回 not\_applicable。这两种路径必须区分。

### 8.2 公共外层

```
```yaml
run_id: ...
case_id: ...
scenario_id: policy_search
axis_id: boundary
type_id: carryability
status: succeeded
reason_code: null
reason: null
config_fingerprint: ...
implementation_fingerprint: ...
input_refs: [...]          # 本次输入及上游结果引用
material_snapshot_refs: [...]
started_at: ...
finished_at: ...
usage: {...}
output: {...}              # 通过该 adapter output_schema 验证
execution_trace_ref: ...
```
```

失败时 output 为空，不能把未通过校验的结果当有效业务结论。必要诊断与未完成证据存入执行记录。

新版轴1保留其期望及 assessment，新版轴2保留 placements 和错误细节。旧轴2的部分条目失败语义要原样记录：新外层可将整个轴标 failed，但保留方法专属部分结果作为诊断，不能静默称整轴成功。



轴1输出直接由运行器保存，不要求轴2重传，也不因轴2失败而丢失。每个轴提交自己的结果，运行器发布已校验输出，其他轴只能读取不能改写。

不跨轴生成未经定义的总分。不同场景即使同类型，其刻度和标准也可能不同，不默认混合统计。

## 9. 原轴复制与基线比较

### 9.1 复制原则

- 固定一个明确的旧代码与配置快照作为参照。
- 列清 system prompt、用户上下文、工具、调用设置、重试、自检、程序映射及失败语义。
- 复制轴专属实现到新 adapter，公共设施继续复用。
- 不在复制时改进措辞、替换判断算法或删除看似多余的业务约束。
- Adapter 协议要求的输入/结果封装变化单独列为差异。

现有 ProjectJudge/JudgeExecution 已有执行策略边界，可借鉴其“公共收尾、内部方法可替换”的模式。新 adapter 不强行继承要求返回 JudgeResult 的主轴接口，避免其他类型被主轴语义限制。

### 9.2 试验入口

提供独立的扩展轴试验服务入口，并由资料页试验操作调用；可以复用项目已有 CLI 命令组织方式提供同等能力。具体 API/命令名在实施时遵循现有路由规范，不增加另一套运行平台。

试验必须明确指定场景和已选样本。初期优先使用冻结的真实回答，不为比较重复调用被测服务。生产入口不调用扩展 runner，无隐式双跑或自动切换。

对照工具分别调用旧链路和新链路，读取相同业务输入及资料。对照工具可以依赖两套入口，扩展 runner 本身不得依赖旧判定。

## 10. 效果与协议验收

### 10.1 两层效果验证

1. 方法层：adapter 是否能可靠执行该类判断。
2. 场景层：当前描述、刻度和资料是否足以准确评本场景。

方法通过不等于所有场景自动通过。引用存在不等于引用支持结论，流程多阶段不等于效果更好。

### 10.2 首版验收项

| 类别   | 验收内容                                |
|------|-------------------------------------|
| 生产隔离 | 未使用试验入口时，不新增模型调用、不改变旧结果或配置          |
| 复制等价 | 用确定性替身验证相同证据输入、关键 prompt、映射、自检与失败处理 |
| 真实效果 | 冻结样本上比较期望、assessment、归位、引用及证据不足情况   |
| 依赖   | 新轴2只消费新轴1；独立轴无须等待；循环配置被拒绝           |
| 隔离   | 输入未连接不注入；一个轴失败不阻止无依赖轴               |
| 结果   | 轴1结果独立留存；下游失败不能覆盖上游；无校验通过的输出不发布     |
| 预算   | 多阶段、工具与重试共同计数；取消或超时后不发布迟到结果         |
| 可追查  | 能找回配置、资料、实际模型端点、工具回执及阶段结果           |

真实模型可能波动，不能要求逐字相同。记录重复运行差异、关键误判、证据不足比例及成本，并人工审查有业务影响的分歧。验收数据需要覆盖成功、失败、边界、缺资料、超时、无待归位期望和部分归位失败。

模型路由切换的实际端点需要记录；不能把模型变化造成的差异直接归因于 adapter。生产切换不属于本次验收的自动后续动作。

## 11. 实施顺序与首版非目标

1. 固定旧链路参照，整理实际输入、方法和结果契约。
2. 实现最小协议、场景存储、输入连接及运行器。
3. 复制新版轴1、新版轴2，跑通冻结单样本端到端链路。
4. 补全失败、依赖、预算和记录验证，完成新旧对照。
5. 接资料页类型选择、业务定义、输入依赖和试验入口。
6. 接结果明细与对照展示；验证首版质量后再接其他 adapter。

首版不做：生产启用、旧实现删除、通用 workflow 平台、DAG 画布编辑器、用户上传可执行代码、任意数据路径表达式、跨场景继承、跨轴总分、自动优化 prompt 或运行中自改方法。

## 12. 相对讨论稿的收敛

- 轴定义不再等于 prompt：实例选类型，类型拥有真实方法。
- 依赖不写死为“轴1之后”：inputs 建立数据依赖，after 只表达完成顺序。
- 连接保存为结构化 binding，不使用任意字符串路径。
- 新版轴1/轴2都属于扩展体系；旧链路只承担生产与比较基线。
- 启用表示参与试验，不表示生产切换；新实例默认关闭。
- 不给所有类型强加统一刻度或统一无法判断值；固定执行状态，保留方法业务语义。
- 不把所有结果经轴2转发：每个轴独立落结果。
- 场景配置和方法实现分别验收，通用协议不能代替判定效果验证。

## 13. 代码依据与配套图稿

本设计基于本次阅读的实际代码，未将历史设计文档当作当前实现：

- ``impl/core/judge_protocol.py``：公共入口、执行策略选择、结果收尾。
- ``impl/core/judge_execution.py``：JudgeExecution / SinglePassJudgeExecution。
- ``impl/core/judge.py``：现有主轴上下文、判定、自检与派生。
- ``impl/core/capability_carrier.py``：逐期望处理、归位和报告。
- ``impl/projects/llm_probe/text_carrier.py``：文本边界判定、资料工具、引用核验与重试。
- ``impl/projects/llm_probe/judge.py``：llm\_probe 主轴特有上下文。
- ``impl/core/llm_client.py``：Agno Agent、结构化调用、工具预算及上下文记录。
- ``impl/projects/llm_probe/material_tools.py``：受目录限制的资料检索工具。

配套概览图：``tmp/eval-axis-internal.html``。图仅表达归属和主连接；协议细节以本文为准。图中轴1结果经轴2方向展示是简化关系，本文明确运行器独立保存每个轴的结果。

## 14 用户原始需求（按原文收录）

以下内容按用户原话收录，仅作为本方案的需求依据，不作为方案正文；文字未做改写、润色或归纳。

### 14.1 交接文件中的原话

> 我发现当前评测系统有一个弱点，虽然主轴确实是最重要之一，但是我们评测的时候，很多场景其实可能需要评估更多的能力，特别是可能没有明确正确标准的问答类场景，所以我希望，除去轴1（即固定的主轴）和轴2（默认的一个重要轴）外，我希望还可以允许用户扩展更多自定义的评估能力轴（最基本的就是比方流畅度、真实性方面的轴），而且轴2应该也属于该体系中（我们尽量不动轴1，当前做的还是挺好的），我们想想这件事情该怎么做。我有一个不成熟的想法，不知道是否合适，就是可以考虑搞定轴1后，同步对各个其他轴进行评估，可以视情况依赖于轴1，也可以完全独立，或者就敢干脆做成一个dag依赖图那种，用户可以对轴间关系进行配置（类似默认轴2就可以依赖轴1），然后还要考虑什么情况下触发轴&哪些样本触发轴（类似轴2应该是轴1nf的才出发&样本信息本身不需要轴1的信息输入？），总之我们好好考虑下，要使得这个东西可配置可使用可自定义，相关配置可以再资料页完成（因此也需要再资料页加一些新模块之类），最后是最重要的效果问题，要看看怎么能确保扩展能力轴做好。你考虑下这个事情应该怎么做，先想想，然后跟我对下，先别动任何东西



> 我不太确定，你的方案看着就感觉很复杂。你先安装这个skill: <https://mp.weixin.qq.com/s/jq5ClGx8TzF19Q4B5P21Dg>，然后画个架构图给我看下

> eval\_axes里面具体是什么东西？我理解最好是跟轴1轴2再llm\_probe前端里面填的东西差不多输入量这种，核心就是一个prompt描述，然后可以应用自由资料里面东西（选择权在用户）

> 我比较关心的一个点是，之前的轴1轴2，是否确实只有描述里面的东西，外层没有别的吗？机制/prompt/啥？一方面是之前的构建有没有围绕轴1/轴2写，二方面是轴1轴2除了prompt之外难道没有任何特定的处理机制吗？（不是指通用的而是就指这两个轴特定的机制？比如流程之类）

> 你基于更新的理解，重新按照archify画一版架构图，能体现里面有啥参数的

## 14.2 后续讨论中的原话

> 你上面的大方向我认同。我补充下，还有一个关键点在于，各个轴的构建逻辑的差异性可允许，现在轴1轴2的执行差异性是可允许的吗，我不是说类似门禁这种，而是在于确保提高效果上，比如通过不同的流程实现，这种现在是否有差异性，然后新的扩展轴是否又能有这种差异性，描述指定固然是一个关键点，但是我觉得不同轴做好judge的关键可能是不一样的，允许差异性和灵活性和怎么控制协议框架和怎么保证效果都是挺重要的，这个是设计上需要考虑，我不希望最后各个轴的差异就只是一个prompt的差异（如果是初始prompt不一样但是探索流程Ai有自助性不会受限我觉得倒也能接受？我不太确定），你考虑下这些问题。关键在于执行方法的差异性这个其实不是只吃描述prompt而是会吃流程的，流程固定带来的问题是固化上线，流程不固定的问题是漂移judge不统一，而且还要考虑我们的实现框架agno 内核+外面的一些东西，能不能做好，总之要先好好想想以及看看我们架构，看看理想的实施方案是什么。

> 我感觉其实是不是可以这样的，一类轴用一套adapter，但是轴上根据各场景的定义是可以差异化的（资料页定义），而是轴的启用也是各场景以及选择，这样的话实现就变成一类轴的实现，然后各场景启用轴可以写自己的描述这种，这个其实跟当前轴1轴2的逻辑其实更接近吧，只是以后新的扩展轴通过adapter的方式搞来允许差异性？这样的话我们要考虑下协议应该怎么搞。

> 我建议你轴1和轴2按照新的逻辑来搞一遍，但是暂时先不启用，先以原来的为主（当然重新搞应该也是以复制逻辑为主我理解）正好到时候可以验证下新的扩展轴逻辑效果，然后这样的话你所有逻辑就在当前扩展评估轴里面解决就好了，就没有那么多外部的依赖，就会干净些。

> 你再画一下架构图，这样的话你可以少写一些乱七八糟的外部的东西对吧。

> 各个adapter之前的协议，连接，执行时间，依赖，信息等等获取咋搞？我看你这个架构图感觉没体现出来。

> 不对啊。我感觉你这样好像都相当于没写啥，比如轴类型定义哪几个字段，怎么填在哪里填，轴场景写什么，依赖/位置之类的在哪里怎么写，你写出来啊。

> 算了，你先写下我们完整的方案文档，感觉你还是有点乱